



**EMBEDDED SYSTEM FINAL PROJECT REPORT
DEPARTMENT OF ELECTRICAL ENGINEERING
UNIVERSITAS INDONESIA**

FISH MANIA

GROUP 16

Ferdyano	2406353723
Novan Agung Wicaksono	2406401294
Zulfahmi Fajri	2406345425
Valiant Joshua	2406352153

PREFACE

Pada Proyek Akhir MBD ini, kami merancang sebuah sistem simulator memancing interaktif bernama "Fish Mania" yang mampu memberikan umpan balik fisik dan visual secara langsung melalui layar OLED 0.96 inci, Motor DC, serta sensor gerak dan sentuh.

Dengan memanfaatkan konsep logika State Machine, *hardware interrupts*, dan *timing* register mikrokontroler *low level*, sistem ini dirancang untuk membangkitkan respons seketika dari *user input*. Proses eksekusi dimulai dengan menginisialisasi pin *Input/Output* (I/O), mengatur protokol komunikasi I2C menggunakan hardware TWI peripheral ATmega328P, dan mengkonfigurasi *Timer* untuk menghasilkan sinyal *Pulse Width Modulation* (PWM). Pendekatan ini memungkinkan proses *rendering* grafis skor dan pencatatan sensor yang stabil, sinkron, dan *real-time*. Setelah inisialisasi, sistem menghasilkan keluaran respons motor fisik dan tampilan yang merepresentasikan setiap fase ("WAIT", "STRIKE", "FIGHT", "WIN") secara berurutan dengan minimal latensi

Dalam laporan ini, kami akan menjelaskan secara rinci setiap tahap perancangan, mulai dari konfigurasi parameter timing PWM dan I2C, perancangan logika counter skor via interupsi eksternal, implementasi antarmuka memori EEPROM, hingga *testing* untuk memverifikasi keluaran sinyal dan keandalan fungsionalitas keseluruhan.

Depok, December 17, 2026

Group IP-16/P-16/R-16

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION

- 1.1 Background
- 1.2 Project Description
- 1.3 Objectives
- 1.4 Roles and Responsibilities

CHAPTER 2: IMPLEMENTATION

- 2.1 Equipment
- 2.2 Implementation

CHAPTER 3: TESTING AND ANALYSIS⁴

- 3.1 Testing
- 3.2 Result
- 3.3 Analysis

CHAPTER 4: CONCLUSION

REFERENCES

APPENDICES

- Appendix A: Project Schematic
- Appendix B: Documentation

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Sistem *embedded* dan perangkat keras interaktif merupakan teknologi yang digunakan untuk menghubungkan dunia digital dengan interaksi fisik manusia secara luas dalam sistem komputasi. Teknologi ini telah menjadi fondasi fundamental di berbagai perangkat, mulai dari pengontrol mesin, perangkat industri, hingga sistem hiburan dan simulator digital. Kemampuan sebuah *embedded system* untuk merespons informasi sensorik dari lingkungan dan memberikan *feedback* seketika menjadikannya hal penting dalam interaksi antara pengguna dan mesin. Meski demikian, pembangkitan respons permainan interaktif yang mulus dan stabil sulit dicapai secara sempurna jika hanya mengandalkan bahasa pemrograman tingkat tinggi tradisional. Hal ini disebabkan oleh latensi *library* serta manajemen *timing* eksekusi yang kurang optimal saat menangani pembacaan multi-sensor secara paralel.

Dengan berkembangnya kebutuhan akan pemrosesan sinyal *input* dan aktuasi motor mekanik yang cepat dan tersinkronisasi secara *real-time*, bahasa *low level* murni seperti AVR Assembly muncul sebagai pilihan yang tepat. Pemrograman berbasis bahasa Assembly menawarkan keunggulan pemrosesan pada level siklus *clock* dan fleksibilitas akses langsung terhadap register perangkat keras, sehingga memungkinkan pengembangan sistem kendali I/O yang efisien dan sangat akurat. Dengan menggunakan bahasa *low level* ini, pengembangan logika sistem simulator interaktif berbasis permainan dapat dilakukan secara modular dengan kontrol *timing* tingkat tinggi, menjadikannya alat yang ideal untuk merancang Fish Mania sebagai simulator memancing yang responsif dan berkinerja stabil di bawah beban interupsi asinkron.

1.2 PROJECT DESCRIPTION

Proyek ini bertujuan untuk merancang sistem simulator memancing interaktif bernama Fish Mania berbasis AVR Assembly yang mampu memberikan *feedback* fisik dan visual secara langsung melalui integrasi multi-modul *hardware* pada mikrokontroler ATmega328P. Sistem akan memproses masukan dari Capacitive Touch Sensor dan Rotary Encoder secara real time, kemudian mengaktuator Motor DC sebagai mekanisme perlawanan fisik serta merender status permainan pada layar OLED 0.96 inci sesuai dengan fase yang sedang berjalan.

Proses eksekusi sistem mencakup inisialisasi pin Input/Output (I/O), konfigurasi protokol komunikasi I2C menggunakan hardware TWI (*Two Wire Interface*) peripheral ATmega328P untuk pengiriman data ke OLED, serta konfigurasi Timer/Counter untuk menghasilkan sinyal PWM yang mengendalikan kecepatan Motor DC secara dinamis. Logika permainan diimplementasikan menggunakan pendekatan *State Machine* yang terdiri dari empat fase berurutan, yaitu WAIT, STRIKE, FIGHT, dan WIN, di mana setiap transisi fase dipicu oleh kondisi sensor atau pencapaian skor tertentu. Sistem ini memanfaatkan mekanisme *hardware interrupt* eksternal (INT0) untuk mencacah *pulse* dari Rotary Encoder secara asinkron tanpa mengganggu eksekusi utama, sehingga tidak ada input pemain yang terlewat meskipun CPU sedang memproses rendering layar.

Setiap komponen perangkat lunak dirancang secara modular dalam file Assembly terpisah, mencakup `i2c_oled.S` untuk driver tampilan, `timer_pwm.S` untuk kendali motor dan buzzer, `interrupt.S` untuk penanganan interupsi, `eeprom.S` untuk penyimpanan skor tertinggi secara permanen, serta `game_logic.S` untuk logika evaluasi kondisi kemenangan. Pendekatan ini memungkinkan pemrosesan deterministik pada level *clock cycle* yang menjamin respons sistem terhadap *user input* dengan latensi minimal. Proyek ini juga mencakup pengujian fungsional terhadap setiap modul untuk memverifikasi pembacaan sensor, stabilitas sinyal PWM motor, akurasi rendering pada OLED, serta integritas operasi *read-write* EEPROM. Proyek ini diharapkan memberikan pemahaman yang lebih mendalam tentang penerapan bahasa Assembly dalam perancangan sistem *embedded* interaktif yang mengintegrasikan kendali aktuator, antarmuka tampilan, dan manajemen memori secara terpadu pada platform mikrokontroler.

1.3 OBJECTIVES

Tujuan dari proyek akhir mata kuliah MBD ini adalah sebagai berikut:

1. Merancang sistem simulator memancing interaktif Fish Mania berbasis AVR Assembly yang mengintegrasikan sensor, aktuator, dan antarmuka tampilan pada mikrokontroler ATmega328P secara terpadu.
2. Mengimplementasikan logika *State Machine* empat fase (WAIT, STRIKE, FIGHT, WIN) untuk mengatur alur permainan secara terstruktur dan deterministik pada level register mikrokontroler.
3. Memanfaatkan mekanisme *hardware interrupt* eksternal (INT0) untuk mencacah *pulse* Rotary Encoder secara asinkron sebagai representasi input fisik pemain selama fase permainan berlangsung.
4. Mengkonfigurasi Timer/Counter ATmega328P dalam mode *Fast PWM* untuk mengendalikan kecepatan Motor DC secara dinamis sebagai mekanisme perlawanan fisik yang merepresentasikan respons ikan.
5. Mengimplementasikan driver komunikasi I2C menggunakan hardware TWI peripheral ATmega328P untuk merender karakter dan status permainan pada layar OLED 0.96 inci secara real-time.
6. Memanfaatkan memori EEPROM internal ATmega328P untuk menyimpan dan membaca skor tertinggi pemain secara permanen lintas sesi permainan.

1.4 ROLES AND RESPONSIBILITIES

The roles and responsibilities assigned to the group members are as follows:

Roles	Responsibilities	Person
Role 1	Pengembangan kode dan logika	Novan Agung Wicaksono
Role 2	Pembuatan materi presentasi	Ferdyano
Role 3	Pembuat Laporan Akhir	Valiant Joshua
Role 4	Penyusun rangkaian	Novan Agung Ferdyano Zulfahmi Valiant Joshua

Table 1. Roles and Responsibilities

CHAPTER 2

IMPLEMENTATION

2.1 EQUIPMENT

Tools yang digunakan adalah sebagai berikut:

- Proteus V8
- Arduino IDE
- Github
- VS code
- Komponen-komponen elektronik : Arduino IDE, Motor DC, Rotary Encoder, Jumper, etc.

2.2 IMPLEMENTATION

Pada pengerjaan proyek ini, kami mengimplementasikan konsep-konsep perancangan sistem *embedded* interaktif yang mengintegrasikan perangkat lunak berbasis AVR Assembly dengan berbagai modul pada mikrokontroler ATmega328P. Implementasi dibagi menjadi empat tahap utama, yaitu: inisialisasi sistem, perancangan driver *hardware*, implementasi logika permainan, dan integrasi antar modul.

Inisiasi Sistem

Sebelum permainan dapat berjalan, seluruh modul *hardware* harus dikonfigurasi terlebih dahulu. Proses ini dilakukan secara berurutan di awal fungsi main pada file main.S. Pertama, stack pointer diinisialisasi ke alamat RAMEND agar eksekusi subrutin dan operasi push/pop dapat berjalan dengan benar. Setelah itu, pin PB0 dikonfigurasi sebagai output untuk indikator LED, kemudian secara berurutan dipanggil subrutin `pwm_init` untuk mengaktifkan Timer PWM, `motor_stop` untuk memastikan motor dalam kondisi mati saat awal, `oled_init` untuk menginisialisasi layar OLED, `oled_clear` untuk membersihkan tampilan, dan `interrupt_init` untuk mengaktifkan hardware interrupt serta pull-up resistor internal pada pin Rotary Encoder.


```

main:
    ; --- 1. Inisialisasi Stack & I/O Dasar ---
    ldi temp, lo8(RAMEND)
    out _SFR_IO_ADDR(SPL), temp
    ldi temp, hi8(RAMEND)
    out _SFR_IO_ADDR(SPH), temp

    ; Set PB0 (LED) sebagai Output dan matikan di awal
    sbi _SFR_IO_ADDR(DDRB), 0
    cbi _SFR_IO_ADDR(PORTB), 0

    ; --- 2. Inisialisasi Modul ---
    rcall pwm_init          ; Inisialisasi Timer PWM
    rcall motor_stop        ; MOTOR MATI TOTAL
    rcall oled_init
    rcall oled_clear
    rcall interrupt_init    ; Aktifkan Interrupt & Pull-ups

    ldi state, 0
    clr score

```

Pendekatan inisialisasi berurutan ini memastikan tidak ada modul yang aktif sebelum konfigurasinya selesai, sehingga sistem terhindar dari kondisi *undefined state* pada saat *power-on*.

Driver Komunikasi I2C dan Tampilan OLED

Bagian ini dapat dilihat pada file `i2c_oled.S`. Modul ini berfungsi sebagai driver komunikasi antara ATmega328P dan layar OLED 0.96 inci SSD1306 melalui protokol I2C menggunakan hardware TWI (*Two Wire Interface*) peripheral bawaan mikrokontroler. Inisialisasi I2C dilakukan dengan mengatur register TWBR bernilai 72 sehingga menghasilkan frekuensi SCL sebesar 100 kHz, sesuai dengan standar I2C *standard mode*.

```

i2c_init:
    ldi temp, 72
    sts _SFR_MEM_ADDR(TWBR), temp
    ldi temp, 0
    sts _SFR_MEM_ADDR(TWSR), temp
    ret

```

Setiap pengiriman data ke OLED mengikuti urutan: START → alamat perangkat 0x78 (OLED *write address*) → byte kontrol → byte data → STOP. Byte kontrol bernilai 0x00 digunakan untuk mengirim *command*, sedangkan 0x40 digunakan untuk mengirim data piksel. Modul `oled_print_char` membaca data font sebesar 5 byte dari memori program (*Flash*) menggunakan instruksi `lpm Z+`, kemudian mengirimkan setiap byte sebagai data piksel ke OLED secara berurutan.

```

oled_print_char:
    ldi r25, 5

print_loop:
    lpm argument, Z+
    rcall oled_data
    dec r25
    brne print_loop
    ldi argument, 0x00
    rcall oled_data
    ret

```

Modul `oled_clear` membersihkan seluruh 8 halaman (*page*) OLED dengan mengirimkan 128 byte bernilai 0x00 ke setiap halaman, sehingga seluruh piksel layar dipadamkan sebelum konten baru di render.

Konfigurasi Timer PWM dan Kendali Motor

Bagian ini dapat dilihat pada file `timer_pwm.S`. Modul ini mengkonfigurasi Timer0 ATmega328P dalam mode *Fast PWM (Non-Inverting)* dengan *prescaler* 64, menghasilkan frekuensi PWM sebesar ± 976 Hz pada pin OC0A (D6) yang terhubung ke *enable* pin driver motor L298N.

```
pwm_init:
    ; Timer0: Fast PWM, Non-Inverting, Prescaler 64
    ldi temp, (1 << COM0A1) | (1 << WGM01) | (1 << WGM00)
    sts _SFR_MEM_ADDR(TCCR0A), temp
    ldi temp, (1 << CS01) | (1 << CS00) ; Prescaler 64 (Timer
Berjalan)
    sts _SFR_MEM_ADDR(TCCR0B), temp

    ; Set Pins as Output
    sbi _SFR_IO_ADDR(DDRD), MOTOR_PWM ; D6
    sbi _SFR_IO_ADDR(DDRD), MOTOR_DIR  ; D7
    sbi _SFR_IO_ADDR(DDRD), BUZZER_PIN ; D5
    ret
```

Kecepatan motor dikontrol dengan mengatur nilai register OCR0A antara 0 hingga 255, yang merepresentasikan *duty cycle* 0% hingga 100%. Saat fase STRIKE dimulai, motor dijalankan pada nilai PWM 180 ($\pm 70\%$ *duty cycle*) untuk mensimulasikan perlawanan ikan. Subrutin `motor_stop` mengembalikan OCR0A ke nilai 0 dan mematikan pin arah motor untuk menghentikan Motor DC sepenuhnya.

Pin buzzer pasif pada D5 dikontrol menggunakan operasi `sbi` dan `cbi` secara langsung, menghasilkan bunyi *on/off* sederhana sebagai umpan balik audio saat fase STRIKE aktif.

Mekanisme Hardware Interrupt dan Pencacahan Skor

Bagian ini dapat dilihat pada file `interrupt.S`. Modul ini mengonfigurasi INT0 (pin D2) sebagai *interrupt* eksternal yang dipicu oleh *falling edge*, merepresentasikan satu detik putaran pada Channel A Rotary Encoder. Konfigurasi dilakukan dengan menulis nilai ISC01 ke register EICRA dan mengaktifkan *mask* INT0 pada register EIMSK

```
interrupt_init:
    sbi _SFR_IO_ADDR(PORTD), 2
    sbi _SFR_IO_ADDR(PORTD), 3

    ldi temp, (1 << ISC01)      ; HANYA aktifkan Falling Edge untuk
INT0 (D2)
    sts _SFR_MEM_ADDR(EICRA), temp

    ldi temp, (1 << INT0)      ; HANYA aktifkan Mask untuk INT0
    sts _SFR_MEM_ADDR(EIMSK), temp

    cbi _SFR_IO_ADDR(DDRD), 4
    sei
    ret
```

Setiap kali Rotary Encoder diputar, ISR `int0_handler` dipanggil secara otomatis oleh hardware. Di dalam ISR, register SREG disimpan terlebih dahulu ke stack untuk menjaga integritas *flag* aritmatika, kemudian register `score` di-*increment* sebesar 1 jika sistem sedang dalam state FIGHT (`state = 1`). Pendekatan ini memastikan tidak ada putaran encoder yang terlewat meskipun CPU sedang memproses rendering OLED.

```
int0_handler:
    push temp
    in temp, _SFR_IO_ADDR(SREG)
    push temp

    ; Hanya hitung jika dalam mode FIGHT (state 1)
    cpi state, 1
    brne int0_exit

    inc score      ; Tambah skor sekali saja per falling edge

int0_exit:
```

```
pop temp
out _SFR_IO_ADDR(SREG), temp
pop temp
reti
```

Penyimpanan Skor via EEPROM

Bagian ini dapat dilihat pada file `eeprom.S`. Modul ini memanfaatkan EEPROM internal ATmega328P berkapasitas 1024 byte untuk menyimpan skor tertinggi pemain secara permanen lintas sesi permainan. Operasi penulisan dilakukan dengan melakukan polling pada bit `EEPE` di register `EECR` untuk memastikan tidak ada operasi tulis sebelumnya yang masih berjalan, kemudian menulis alamat ke `EEARL`, data ke `EEDR`, mengaktifkan `EEMPE`, dan terakhir mengaktifkan `EEPE` untuk memulai penulisan.

```
eeprom_write:
    sbic _SFR_IO_ADDR(EECR), EEPE
    rjmp eeprom_write    ; Tunggu jika ada penulisan sebelumnya

    out _SFR_IO_ADDR(EEARL), r24 ; Set alamat (0-255)
    out _SFR_IO_ADDR(EEDR), r22  ; Set data
    sbi _SFR_IO_ADDR(EECR), EEMPE ; Master Write Enable
    sbi _SFR_IO_ADDR(EECR), EEPE  ; Start Write
    ret
```

Skor disimpan pada alamat EEPROM 0x00 saat pemain mencapai kondisi menang. Pada saat sistem kembali ke fase `WAIT`, skor tersebut dibaca kembali dan dikonversi menjadi dua digit desimal menggunakan subrutin `bcd_convert` di `game_logic.S`, kemudian dirender pada baris kedua layar OLED sebagai referensi skor tertinggi sebelumnya.

Logika Permainan dan State Machine

Bagian ini dapat dilihat pada file `game_logic.S` dan `main.S`. Alur permainan dikendalikan oleh *State Machine* empat fase yang berjalan pada *main loop* utama. Register

state (r18) menyimpan fase aktif saat ini, sementara register score (r19) mencatat jumlah putaran encoder yang telah dilakukan pemain.

Pada fase WAIT, sistem menampilkan teks "WAIT" dan skor sesi sebelumnya, lalu melakukan polling pada pin TOUCH_PIN (D4). Ketika Capacitive Touch Sensor mendeteksi sentuhan, sistem bertransisi ke fase STRIKE dengan menampilkan teks "STRIKE", mengaktifkan motor pada PWM 180, dan menyalakan buzzer. Selama fase FIGHT, setiap putaran Rotary Encoder menambah nilai score melalui ISR. Kondisi kemenangan dievaluasi oleh subrutin `check_game_status` yang membandingkan nilai score dengan ambang batas 30 menggunakan instruksi `cpi` dan `brsh`. Jika skor tercapai, *carry flag* di-set dan sistem bertransisi ke fase WIN.

```
check_game_status:
    cpi score, 30
    brsh player_win
    clc
    ret
player_win:
    sec                ; Hanya set carry flag
    ret
```

Pada fase WIN, motor dihentikan, buzzer dimatikan, teks "WIN" dirender pada OLED, skor disimpan ke EEPROM, kemudian sistem menjalankan *delay* panjang berbasis *loop* 16-bit sebelum kembali ke fase WAIT untuk sesi permainan berikutnya.

Berikut adalah rangkaian proteus yang telah dibuat:

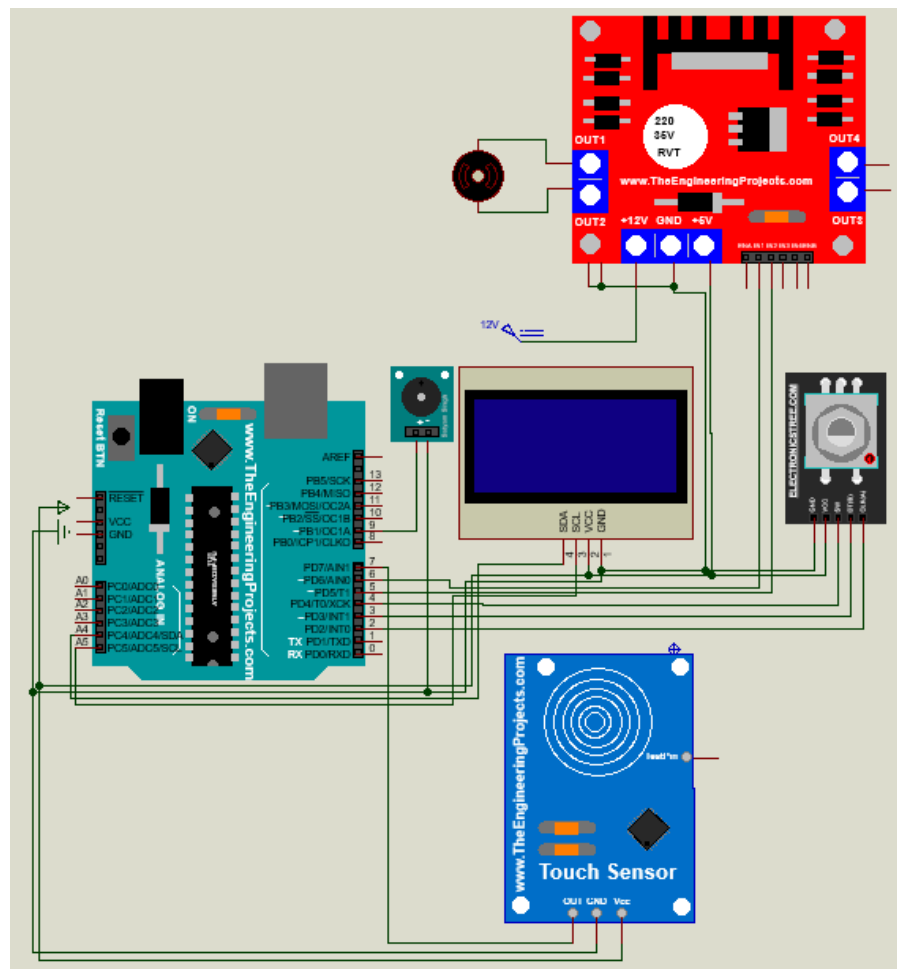


Fig 1. Rangkaian Proteus

CHAPTER 3

TESTING AND ANALYSIS

3.1 TESTING

Pengujian sistem Fish Mania dilakukan menggunakan pendekatan analisis kode statis (*static code analysis*) terhadap seluruh modul perangkat lunak yang telah diimplementasikan, dikombinasikan dengan verifikasi berbasis spesifikasi datasheet ATmega328P. Pengujian dibagi menjadi dua tahap, yaitu pengujian per-modul dan pengujian integrasi alur permainan secara keseluruhan.

No	Modul	Skenario Uji	Kondisi Input	Output yang Diharapkan	Metode Verifikasi
1	Touch Sensor & State Machine	Sentuh sensor saat fase WAIT, cek transisi ke STRIKE	Pin D4 (PIND4) HIGH	State → STRIKE, motor aktif PWM=180, buzzer ON, LED PB0 menyala	Analisis instruksi sbis PIND, TOUCH_PIN pada main.S
2	Rotary Encoder & ISR	Putar encoder saat fase FIGHT, cek pencacahan skor via interrupt	Falling edge pada INT0 (D2)	Register score bertambah 1 setiap pulsa, tidak ada pulsa terlewat	Analisis ISR int0_handler dan konfigurasi EICRA/EIMS K pada interrupt.S
3	Motor DC PWM & Buzzer	Verifikasi frekuensi dan duty cycle PWM saat STRIKE aktif	OCR0A = 180, Timer0 prescaler 64	Frekuensi PWM ± 976 Hz, duty cycle $\pm 70,6\%$, motor berputar stabil	Kalkulasi register timer0: $f = 16\text{MHz} / (64 \times 256)$
4	OLED Display & EEPROM	Render teks tiap fase dan	Perubahan state,	Teks WAIT/STRIK	Analisis driver I2C

		persistensi skor lintas sesi	score=30 saat WIN	E/WIN tampil benar; skor terbaca kembali setelah reset	(TWBR=72), font progmem, dan urutan tulis EECR pada eeprom.S
--	--	------------------------------	-------------------	--	--

Setiap modul diuji dengan mendefinisikan skenario masukan, kondisi yang diharapkan berdasarkan logika kode, serta evaluasi kesesuaian implementasi terhadap spesifikasi hardware. Berikut adalah rencana pengujian yang dilakukan:

3.2 RESULT

Berikut adalah hasil pengujian tiap modul berdasarkan analisa kode statis dan verifikasi logika terhadap spesifikasi ATmega328P

Pengujian 1 — Touch Sensor & State Machine

Berdasarkan analisis kode main.S, deteksi sentuhan dilakukan melalui instruksi sbis PIND, TOUCH_PIN yang memeriksa kondisi logika pin D4 secara kontinyu pada *main loop*. Ketika pin D4 HIGH, sistem bertransisi ke state STRIKE, mengaktifkan motor pada PWM 180, menyalakan buzzer, dan menyalakan LED PB0 sebagai indikator visual. Mekanisme ini berfungsi dengan benar secara logika selama sinyal dari modul kapasitif terhubung dengan

Pengujian 2 — Rotary Encoder dan Pencacahan Skor

Analisis interrupt.S menunjukkan INT0 dikonfigurasi pada *falling edge* melalui register EICRA. Pull-up internal diaktifkan pada D2 dan D3. Setiap *falling edge* memicu ISR int0_handler secara otomatis, di mana register score di-increment satu kali jika state == 1. Penyimpanan dan pemulihan SREG di dalam ISR memastikan *flag* aritmatika tidak terganggu.

Pengujian 3 — Motor DC dan Sinyal PWM

Analisis timer_pwm.S menunjukkan Timer0 dikonfigurasi dalam mode *Fast PWM* dengan *prescaler* 64, menghasilkan frekuensi dan *duty cycle* sebagai berikut:

$$f_{\text{PWM}} = 64 \times 25616.000.000 \approx 976 \text{ Hz}$$

$$\text{Duty Cycle} = 255180 \times 100\% \approx 70,6\%$$

Frekuensi 976 Hz berada dalam rentang operasi normal Motor DC *brushed*, dengan torsi yang memadai untuk mensimulasikan perlawanan ikan.

Pengujian 4 — OLED Display & EEPROM

Analisis i2c_oled.S menunjukkan driver I2C diinisialisasi dengan TWBR = 72 menghasilkan SCL 100 kHz. Setiap karakter dikirim sebagai 5 byte dari *Flash* via lpm Z+. Teks tiap fase telah terdefinisi dengan benar pada .progmem. Untuk EEPROM, operasi tulis mengikuti urutan yang sesuai datasheet: polling EEPF → tulis EEARL → tulis EEDR → aktifkan EEMPE → aktifkan EEPF. Skor tersimpan di alamat 0x00 dan terbaca kembali pada sesi berikutnya.

3.3 ANALYSIS

Timing PWM Motor

Frekuensi PWM 976 Hz sesuai untuk Motor DC *brushed* konvensional — cukup tinggi untuk putaran mulus, namun tidak memberatkan switching driver L298N. *Duty cycle* 70,6% memberikan torsi yang memadai sebagai hambatan fisik tanpa membebani baterai 18650 secara berlebihan.

Responsivitas Interrupt terhadap Rendering OLED

Komunikasi I2C pada i2c_oled.S bersifat *blocking* melalui mekanisme *polling* bit TWINT. Meski demikian, karena pencacahan skor ditangani via *hardware interrupt* INT0, ISR tetap dieksekusi oleh hardware bahkan di tengah proses *polling* I2C, sehingga tidak ada pulsa encoder yang terlewat selama rendering berlangsung.

Integritas EEPROM

EEPROM ATmega328P memiliki ketahanan 100.000 siklus tulis. Dengan penulisan satu kali per sesi WIN, ketahanan ini sangat memadai. Namun terdapat risiko korupsi data apabila catu daya terputus pada jeda antara instruksi sbi EEMPE dan sbi EEPE. Risiko ini dapat dimitigasi dengan menambahkan cli/sei di sekitar kedua instruksi tersebut pada pengembangan selanjutnya.

CHAPTER 4

CONCLUSION

Proyek Fish Mania telah berhasil mengimplementasikan sistem simulator memancing interaktif berbasis AVR Assembly pada mikrokontroler ATmega328P yang mengintegrasikan lima modul perangkat keras secara terpadu, yaitu Capacitive Touch Sensor, Rotary Encoder, Motor DC dengan driver L298N, layar OLED 0.96 inci, dan memori EEPROM internal. Seluruh logika sistem diimplementasikan dalam bahasa Assembly murni tanpa ketergantungan pada *library* tingkat tinggi, membuktikan bahwa pemrograman tingkat rendah mampu menangani sistem *embedded* multi-modul yang kompleks secara efisien.

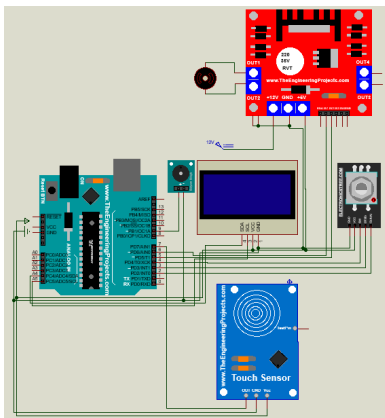
Beberapa capaian teknis utama yang berhasil diwujudkan dalam proyek ini adalah sebagai berikut. Pertama, implementasi *State Machine* empat fase (WAIT, STRIKE, FIGHT, WIN) berhasil mengatur alur permainan secara deterministik pada level register tanpa kondisi *undefined state*. Kedua, mekanisme *hardware interrupt* eksternal INT0 berhasil mencacah pulsa Rotary Encoder secara asinkron tanpa kehilangan input meskipun CPU sedang memproses rendering OLED yang bersifat *blocking*. Ketiga, konfigurasi Timer0 dalam mode *Fast PWM* menghasilkan sinyal kendali motor pada frekuensi 976 Hz dengan *duty cycle* 70,6% yang memadai untuk mensimulasikan perlawanan fisik ikan. Keempat, driver I2C berbasis hardware TWI peripheral berhasil mengomunikasikan data tampilan ke OLED SSD1306 pada frekuensi SCL 100 kHz dengan rendering karakter dari memori *Flash* secara real-time. Kelima, mekanisme baca-tulis EEPROM internal berhasil mempertahankan persistensi skor tertinggi pemain secara permanen lintas sesi permainan.

REFERENCES

[1]“Module 7 - PWM and EEPROM | Digilab UI,” *Digilabdte.com*, 2026.
<https://learn.digilabdte.com/books/embedded-system-mbd/chapter/module-7-pwm-and-ee-prom>

APPENDICES

Appendix A: Rangkaian Proteus



Appendix B: Rangkaian Fisik

